

Isolation Levels and MVCC

Chapter 05 - Sub-Chapter 05

SQL Databases Series

Isolation Levels	READ COMMITTED, REPEATABLE READ, SERIALIZABLE
MVCC	Multi-version concurrency, xmin/xmax, snapshots
Spring Isolation	Setting levels per transaction, write skew
VACUUM and Bloat	Dead tuples, autovacuum, XID wraparound

1 Isolation Levels

The four levels and their concurrency anomalies

Perfect isolation (every transaction runs as if alone) is expensive — it requires extensive locking. SQL defines four isolation levels that trade isolation for concurrency. Understanding what anomaly each level allows is critical for choosing the right level for your workload.

Isolation Levels vs Concurrency Phenomena

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Write Skew
READ UNCOMMITTED	P	P	P	P
READ COMMITTED	✓	P	P	P
REPEATABLE READ	✓	✓	P*	P
SERIALIZABLE	✓	✓	✓	✓

P = Possible (can occur) ✓ = Prevented P* = Prevented in PostgreSQL RR (but not SQL standard)

The three concurrency anomalies explained with examples:

```
-- DIRTY READ: reading another transaction's uncommitted changes
-- T1: UPDATE accounts SET balance=0 WHERE id=1; (not yet committed)
-- T2: SELECT balance FROM accounts WHERE id=1; --> sees 0 (dirty!)
-- T1: ROLLBACK; -- balance was never really 0
-- T2 made a decision based on a value that never existed!
-- Prevented by: READ COMMITTED and above

-- NON-REPEATABLE READ: same query returns different results within one transaction
-- T1: SELECT balance FROM accounts WHERE id=1; --> 1000
-- T2: UPDATE accounts SET balance=500 WHERE id=1; COMMIT;
-- T1: SELECT balance FROM accounts WHERE id=1; --> 500 (changed!)
-- T1 sees different values for the same row in the same transaction
-- Prevented by: REPEATABLE READ and above

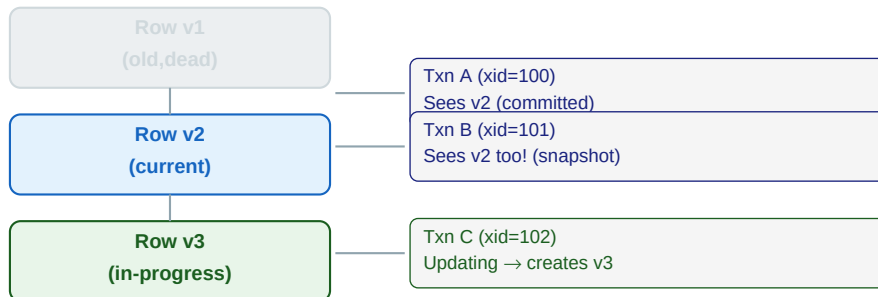
-- PHANTOM READ: a new row appears between two reads in the same transaction
-- T1: SELECT COUNT(*) FROM orders WHERE user_id=42; --> 5
-- T2: INSERT INTO orders (user_id, ...) VALUES (42,...); COMMIT;
-- T1: SELECT COUNT(*) FROM orders WHERE user_id=42; --> 6 (phantom!)
-- Prevented by: SERIALIZABLE
```

2 MVCC — Multi-Version Concurrency Control

How PostgreSQL achieves isolation without blocking

Most databases use locking to enforce isolation — readers wait for writers and vice versa. PostgreSQL uses MVCC: it keeps multiple versions of each row. Readers get a consistent snapshot from the past; writers create new versions. Readers never block writers. Writers never block readers.

MVCC — Multiple Versions of the Same Row



Readers never block writers · Writers never block readers · Each txn sees a consistent snapshot

How MVCC works internally:

```
-- Every row has hidden system columns:
-- xmin: transaction ID that created this row version
-- xmax: transaction ID that deleted/updated this row version (0 if current)
-- ctid: physical location of the row version on disk

-- When you UPDATE a row:
-- 1. The old row version is marked with xmax = current transaction ID
-- 2. A new row version is created with xmin = current transaction ID
-- Both versions exist simultaneously on disk!

-- When you READ:
-- PostgreSQL checks each row's xmin/xmax against your snapshot
-- Your snapshot = set of committed transactions when your statement started
-- You see rows where: xmin is committed and in your snapshot
--                      AND xmax is either 0 or not yet committed in your snapshot

-- Dead tuples: old versions (xmax set) are "dead"
-- VACUUM reclaims dead tuples and frees disk space
-- Without VACUUM, dead tuples accumulate → "table bloat" → slow queries

-- Check dead tuples on a table:
SELECT relname, n_live_tup, n_dead_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
WHERE relname = 'orders';
-- If dead_pct > 10%: VACUUM ANALYZE orders;
```

3 Setting Isolation in Spring

Choosing the right level for each use case

PostgreSQL's default is READ COMMITTED — a good default for most applications. You only need to increase it when specific anomalies cause bugs in your business logic.

```
import org.springframework.transaction.annotation.Transactional;
import org.springframework.transaction.annotation.Isolation;

// READ COMMITTED (PostgreSQL default)
// Use for: most CRUD operations, report generation, dashboard queries
// Prevents: dirty reads
// Allows: non-repeatable reads, phantom reads
@Transactional(isolation = Isolation.READ_COMMITTED)
public List<Order> getDashboardOrders() { ... }

// REPEATABLE READ
// Use for: batch processing that reads same rows multiple times,
//         financial summaries that need consistent numbers throughout
// Prevents: dirty reads, non-repeatable reads
// Allows: phantom reads (but PostgreSQL's SSI prevents most phantoms here too)
@Transactional(isolation = Isolation.REPEATABLE_READ)
public MonthlyReport generateMonthlyReport(YearMonth month) { ... }

// SERIALIZABLE
// Use for: financial transfers, inventory reservation, any "check then act"
//         pattern where two concurrent transactions can produce wrong results
// Prevents: ALL anomalies including write skew
// Cost: more aborts + retries needed when conflicts detected
@Transactional(isolation = Isolation.SERIALIZABLE)
@Retryable(value = CannotSerializeTransactionException.class, maxAttempts = 3)
public void reserveSeat(Long flightId, int seatNumber, Long userId) {
    // Check if seat is available
    Seat seat = seatRepo.findByFlightAndNumber(flightId, seatNumber);
    if (!seat.isAvailable()) throw new SeatUnavailableException();
    // Reserve it
    seat.setUserId(userId);
    seat.setAvailable(false);
    seatRepo.save(seat);
    // SERIALIZABLE prevents two transactions from both seeing "available"
    // and both reserving the same seat!
}
```

Write Skew — the anomaly only SERIALIZABLE prevents:

```
-- Write skew: two transactions each read the same data,
-- make decisions based on it, and write different rows
-- Result: the combined writes violate a constraint that each individually does not

-- Example: doctor on-call system
-- Rule: at least 1 doctor must always be on call
-- Initially: Alice=oncall, Bob=oncall (2 doctors on call)

-- T1 (Alice): SELECT COUNT(*) FROM oncall WHERE on_duty=true; --> 2
--             (Decides it is safe to go off-call since Bob is still on)
--             UPDATE doctors SET on_duty=false WHERE name='Alice';

-- T2 (Bob):   SELECT COUNT(*) FROM oncall WHERE on_duty=true; --> 2
--             (Decides it is safe to go off-call since Alice is still on)
```

```
--          UPDATE doctors SET on_duty=false WHERE name='Bob';

-- Result: BOTH go off-call! 0 doctors on duty. Rule violated.
-- READ COMMITTED and REPEATABLE READ cannot prevent this.
-- SERIALIZABLE detects that T1 and T2 conflict and aborts one.
```

4

VACUUM, bloat, and XID Wraparound

Keeping MVCC healthy in production

MVCC leaves dead row versions on disk after updates and deletes. VACUUM reclaims this space and updates visibility maps. Without regular VACUUM, tables bloat, queries slow down, and eventually PostgreSQL hits the dreaded XID wraparound — a hard stop to prevent data corruption.

```
-- Check if autovacuum is keeping up
SELECT schemaname, tablename,
       last_vacuum, last_autovacuum,
       last_analyze, last_autoanalyze,
       n_dead_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup+n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 10;

-- Manual VACUUM + ANALYZE (safe in production, does not lock table)
VACUUM ANALYZE orders;

-- VACUUM FULL: reclaims disk space but locks table (use in maintenance window)
VACUUM FULL orders;

-- Check XID age -- must stay well below 2 billion
SELECT datname,
       age(datfrozenxid) AS xid_age,
       2000000000 - age(datfrozenxid) AS xids_until_wraparound
FROM pg_database
ORDER BY xid_age DESC;
-- Alert if xid_age > 1.5 billion! Trigger aggressive VACUUM.

-- Tune autovacuum for high-write tables
ALTER TABLE orders SET (
    autovacuum_vacuum_cost_delay = 2,          -- ms, lower = more aggressive
    autovacuum_vacuum_scale_factor = 0.01,     -- vacuum after 1% dead rows
    autovacuum_analyze_scale_factor = 0.005
);
```

Key Rules

- PostgreSQL default isolation is READ COMMITTED -- only change it when you have a specific anomaly to prevent.
- REPEATABLE READ for batch jobs and reports -- same data throughout the transaction.
- SERIALIZABLE + @Retryable for 'check then act' patterns (seat reservation, inventory).
- MVCC means readers never block writers -- but dead tuples accumulate. Monitor n_dead_tup.
- Run VACUUM ANALYZE after large bulk operations (imports, deletes).
- Monitor XID age -- wraparound is catastrophic. Set up alerting at 1.5B XIDs.